

Aprendizagem Industrial em Programador de Sistemas da Informação

Turma - AI PSII 2026/1 V1

Data: 08/10/2026

UC: Criação e Manipulação de Banco de dados

Professor: Lucas de Mattos

RELATÓRIO - DESENVOLVIMENTO E ESTRUTURAÇÃO - BANCO DE DADOS

Leticia Barboza

INTRODUÇÃO

Este relatório apresenta o desenvolvimento de um banco de dados criado utilizando o **MySQL**, por meio do ambiente **Laragon** e com auxílio do **HeidiSQL** para criação, organização e gerenciamento das estruturas.

O banco de dados foi desenvolvido com o objetivo de organizar informações relacionadas a **clientes, fornecedores, produtos e compras**, permitindo armazenar os dados de forma estruturada e realizar consultas de maneira mais rápida e organizada.

Durante o desenvolvimento, foram utilizadas diferentes funcionalidades do banco de dados, como **tabelas, Stored Procedures, Views e Triggers**. Cada uma dessas ferramentas possui uma função específica e contribui para deixar o sistema mais organizado e automatizado.

O projeto também utiliza relacionamentos entre as tabelas. Por exemplo, a tabela de compras possui informações relacionadas aos produtos e aos fornecedores, permitindo consultar quem forneceu determinado produto, qual foi a quantidade comprada e em qual data a compra ocorreu.

2. TIPO DE BANCO DE DADOS UTILIZADO

O banco de dados utilizado no projeto é um **banco de dados relacional**, desenvolvido em **MySQL**.

O MySQL foi escolhido porque trabalha com informações organizadas em **tabelas**, formadas por linhas e colunas. Essas tabelas podem se relacionar umas com as outras por meio de identificadores, como IDs.

No projeto, existem tabelas diferentes para armazenar cada tipo de informação. Dessa forma, os dados ficam separados de acordo com sua finalidade, evitando que todas as informações sejam colocadas em uma única tabela.

Por que utilizar um banco de dados relacional?

O modelo relacional foi escolhido porque é adequado para sistemas que possuem informações que precisam se relacionar.

No projeto, por exemplo:

- um **produto** pode aparecer em várias compras;
- um **fornecedor** pode estar relacionado a várias compras;
- um **cliente** possui suas próprias informações de cadastro;
- Uma **compra** relaciona informações de produtos e fornecedores.

Esse modelo facilita a organização e a consulta das informações.

Outra vantagem é a possibilidade de utilizar comandos **SQL** para inserir, alterar, excluir e consultar dados.

O MySQL também permite utilizar recursos mais avançados, como:

- SELECT;
- INSERT;
- UPDATE;
- DELETE;
- JOIN;
- GROUP BY;
- Views;
- Stored Procedures;

- Triggers.

Esses recursos foram utilizados ao longo do projeto para melhorar o funcionamento do banco.

Ambiente utilizado

Para o desenvolvimento foi utilizado o **Laragon**, que fornece o ambiente necessário para trabalhar com o banco de dados localmente. O **HeidiSQL** também foi utilizado para visualizar as tabelas, executar comandos SQL e verificar os resultados das consultas.

3. TABELAS E DADOS ARMAZENADOS

As tabelas são uma das principais partes do banco de dados. Elas são responsáveis por armazenar as informações de maneira organizada.

No projeto foram utilizadas principalmente as tabelas:

- clientes;
- fornecedores;
- produtos;
- compras.
- Cada uma possui uma função diferente.

3.1 Tabela

CLIENTES

A tabela clientes é responsável por armazenar os dados relacionados aos clientes cadastrados no sistema.

Entre as informações utilizadas estão dados de identificação e contato, como:

- ID do cliente;
- nome;
- bairro;
- telefone.

O ID serve para identificar cada cliente de forma única.

Essas informações também podem ser utilizadas em consultas para encontrar rapidamente os dados de contato de determinado cliente.

Colunas: + Adicionar × Remover ▲ Mover para cima ▼ Mover para baixo									
#	Nome	Tipo de dados	Tamanho/It...	Unsign...	Permitir...	Zero fill	Padrão	Comentário	Colação
1	id	INT		☐	☐	☐	AUTO_INCREME...		
2	nome	VARCHAR	100	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci
3	cnpj_cpf	VARCHAR	18	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci
4	rua	VARCHAR	255	☐	☒	☐	NULL		utf8mb4_general_ci
5	numero	VARCHAR	50	☐	☒	☐	NULL		utf8mb4_general_ci
6	bairro	VARCHAR	100	☐	☒	☐	NULL		utf8mb4_general_ci
7	cep	VARCHAR	10	☐	☒	☐	NULL		utf8mb4_general_ci
8	referencia	VARCHAR	255	☐	☒	☐	NULL		utf8mb4_general_ci
9	telefone	VARCHAR	100	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci

FORNECEDORES

A tabela fornecedores armazena as informações referentes aos fornecedores.

Os fornecedores são importantes porque são responsáveis pelo fornecimento dos produtos utilizados nas compras registradas no banco.

A identificação do fornecedor é utilizada também na tabela compras, permitindo relacionar cada compra ao fornecedor correspondente.

Colunas: + Adicionar ✕ Remover ▲ Mover para cima ▼ Mover para baixo								
#	Nome	Tipo de dados	Tamanho/It...	Unsign...	Permitir...	Zerofill	Padrão	Comentário
1	id	INT		☐	☐	☐	AUTO_INCREME...	
2	nome	VARCHAR	255	☐	☐	☐	Nenhum padrão	
3	cnpj_cpf	VARCHAR	18	☐	☐	☐	Nenhum padrão	
4	rua	VARCHAR	255	☐	☒	☐	NULL	
5	numero	VARCHAR	50	☐	☒	☐	NULL	
6	bairro	VARCHAR	100	☐	☒	☐	NULL	
7	cep	VARCHAR	10	☐	☒	☐	NULL	
8	telefone	VARCHAR	100	☐	☐	☐	Nenhum padrão	

PRODUTOS

A tabela produtos armazena os produtos cadastrados no sistema.

Entre as informações utilizadas estão principalmente:

- ID do produto;
- nome;
- preço.

O ID identifica o produto e pode ser utilizado para relacioná-lo com outras tabelas.

O nome permite identificar qual produto está cadastrado, enquanto o preço representa seu valor.

Essa tabela também está relacionada às compras, pois uma compra pode possuir determinado produto.

Colunas: + Adicionar ✕ Remover ▲ Mover para cima ▼ Mover para baixo								
#	Nome	Tipo de dados	Tamanho/It...	Unsign...	Permitir...	Zerofill	Padrão	Comentário
1	id	INT		☐	☐	☐	AUTO_INCREME...	
2	cod_barra	VARCHAR	100	☐	☒	☐	NULL	
3	nome	VARCHAR	255	☐	☐	☐	Nenhum padrão	
4	descricao	LONGTEXT		☐	☒	☐	NULL	
5	estoque	FLOAT		☐	☐	☐	Nenhum padrão	
6	valor	FLOAT		☐	☐	☐	Nenhum padrão	
7	data_fabricacao	DATETIME		☐	☒	☐	NULL	
8	data_validade	DATETIME		☐	☒	☐	NULL	

COMPRAS

A tabela compras registra as compras realizadas.

Ela possui informações que permitem saber, por exemplo:

código ou ID da compra;
produto comprado;
quantidade;
data da compra;
fornecedor.

A tabela compras é importante porque faz a ligação entre diferentes informações do banco. Por meio dela é possível descobrir qual produto foi comprado, qual foi a quantidade, quando ocorreu a compra e qual fornecedor esteve relacionado à operação.

Colunas: + Adicionar × Remover ▲ Mover para cima ▼ Mover para baixo								
#	Nome	Tipo de dados	Tamanho/It...	Unsign...	Permitir...	Zerofill	Padrão	Comentário
1	id	INT		☐	☐	☐	AUTO_INCREME...	
2	id_fornecedor	INT		☐	☒	☐	NULL	
3	id_produto	INT		☐	☒	☐	NULL	
4	quantidade	FLOAT		☐	☐	☐	Nenhum padrão	
5	codigo	VARCHAR	100	☐	☒	☐	NULL	
6	data_compra	DATETIME		☐	☒	☐	NULL	

Relação entre as tabelas

As tabelas não funcionam de maneira isolada. Elas possuem relações entre si.

A tabela compras, por exemplo, utiliza identificadores relacionados às tabelas produtos e fornecedores. Assim, em uma consulta SQL, é possível utilizar JOIN para juntar informações de várias tabelas.

Isso evita a repetição desnecessária de informações e deixa o banco mais organizado.

4. STORED PROCEDURES

As **Stored Procedures**, ou procedimentos armazenados, são conjuntos de comandos SQL que ficam armazenados dentro do próprio banco de dados.

Elas podem ser utilizadas para realizar tarefas específicas sem que seja necessário escrever todos os comandos novamente sempre que a operação for executada.

No projeto, foi criada uma procedure para realizar a atualização de informações de um produto.

Procedure

atualizar_produto

A procedure possui parâmetros relacionados ao produto:

- p_id;
- p_nome;
- p_preco.

Sua função é localizar um produto pelo seu ID e atualizar seu nome e seu preço.

De forma simplificada, seu funcionamento é:

ID do produto → localizar produto → atualizar nome e preço.

O comando utilizado realiza um UPDATE na tabela produtos, alterando os valores correspondentes ao produto cujo ID foi informado.

Para que a procedure serve?

A utilização dessa procedure facilita a manutenção dos dados.

Sem uma procedure, seria necessário escrever novamente o comando UPDATE sempre que fosse necessário alterar um produto.

Com a procedure, basta informar os parâmetros necessários para realizar a alteração.

Por exemplo, podem ser informados:

- o ID do produto que será alterado;
- o novo nome;
- o novo preço.

Isso torna o processo mais organizado e reduz a repetição de comandos.

Vantagens das Stored Procedures

As procedures apresentam algumas vantagens:

Organização: os comandos ficam armazenados no banco.

Reutilização: a mesma procedure pode ser executada várias vezes.

Praticidade: facilita operações que precisam ser realizadas com frequência.

Padronização: a operação é realizada sempre seguindo a mesma lógica.

No projeto, a procedure de atualização de produtos demonstra como uma tarefa comum pode ser automatizada dentro do banco de dados.

5. VIEWS E OS DADOS APRESENTADOS

As **Views**, ou visões, são consultas armazenadas no banco de dados que funcionam como uma espécie de tabela virtual.

Elas não precisam armazenar novamente os dados. Em vez disso, apresentam informações obtidas a partir das tabelas existentes.

No projeto foram utilizadas Views para facilitar consultas específicas.

5.1 View

vw_clientes_contato

A View vw_clientes_contato apresenta informações de contato dos clientes.

Os dados apresentados são:

- ID;
- nome;
- bairro;
- telefone.

Ela facilita a consulta dessas informações porque não é necessário escrever novamente todo o comando SELECT.

A ideia da View é apresentar somente os dados necessários para uma determinada finalidade.

5.2 View

vw_resumo_fornecedores

A View vw_resumo_fornecedores apresenta um resumo das informações relacionadas aos fornecedores.

Ela utiliza:

- id_fornecedor;
- quantidade de transações;
- volume total de produtos.

Para chegar a esses resultados são utilizadas funções como COUNT() e SUM(). O COUNT() permite contar quantas transações estão relacionadas a determinado fornecedor.

Já o SUM(quantidade) permite calcular o volume total de produtos registrados nas compras.

A consulta utiliza também GROUP BY, agrupando os resultados pelo fornecedor.

Dessa forma, em vez de apresentar todas as compras individualmente, a View apresenta um resumo para cada fornecedor.

5.3 View

vw_compras_grandes_volumes

Outra View utilizada no projeto é a vw_compras_grandes_volumes.

Ela apresenta compras que possuem uma quantidade superior a **50 unidades**.

Entre os dados apresentados estão:

- ID;
- código;
- ID do produto;
- quantidade;
- data da compra.

Essa View facilita a identificação de compras que possuem grandes quantidades de produtos.

Importância das Views

As Views ajudam a deixar o banco mais organizado e facilitam consultas que precisam ser utilizadas com frequência.

Elas também podem deixar os resultados mais fáceis de interpretar, pois permitem selecionar somente as informações importantes para cada situação.

6. TRIGGERS

Os **Triggers**, ou gatilhos, são recursos do banco de dados utilizados para executar automaticamente uma determinada ação quando ocorre um evento em uma tabela. Diferentemente de uma procedure, que normalmente precisa ser chamada para ser executada, o trigger é acionado automaticamente quando a condição definida acontece. No projeto foi utilizado um trigger relacionado à tabela produtos.

Trigger

trg_atualiza_produtos

O trigger trg_atualiza_produtos está associado a uma alteração (UPDATE) realizada na tabela produtos.

Ele utiliza o evento **AFTER UPDATE**, ou seja, sua execução acontece depois que uma alteração é realizada na tabela.

Dessa forma, quando uma informação de um produto é atualizada, o trigger é acionado automaticamente conforme a lógica definida no banco.

Para que os Triggers servem?

Os triggers podem ser utilizados para automatizar tarefas e manter determinadas informações do banco atualizadas.

Eles são úteis principalmente quando uma ação precisa acontecer automaticamente após uma operação.

No projeto, o uso do trigger demonstra como o banco de dados pode executar determinadas tarefas sem que seja necessário realizar manualmente todas as etapas.

Diferença entre Procedure, View e Trigger

Apesar de todos fazerem parte do banco de dados, cada recurso possui uma função diferente:

Procedure: executa uma sequência de comandos quando é chamada. No projeto, é utilizada para atualizar informações de um produto.

View: apresenta dados através de uma consulta armazenada. No projeto, é utilizada para facilitar consultas de clientes, fornecedores e compras.

Trigger: executa automaticamente uma ação quando ocorre determinado evento em uma tabela.

CONCLUSÃO

O desenvolvimento deste banco de dados possibilitou colocar em prática diferentes conceitos de banco de dados relacionais e da linguagem SQL.

A utilização do **MySQL** foi adequada ao projeto porque permite organizar os dados em tabelas relacionadas e realizar consultas de forma eficiente.

As tabelas clientes, fornecedores, produtos e compras são responsáveis pelo armazenamento das principais informações do sistema. Cada uma possui uma função específica e, por meio dos relacionamentos, é possível cruzar os dados entre elas.

As **Stored Procedures** facilitaram a realização de operações, como a atualização dos dados dos produtos. As **Views** permitiram criar consultas específicas e apresentar somente as informações necessárias. Já os **Triggers** demonstraram como determinadas ações podem ser executadas automaticamente após alterações no banco.

Assim, o projeto mostrou que um banco de dados não serve apenas para armazenar informações, mas também pode ser utilizado para **organizar, consultar, resumir e automatizar dados**, tornando o sistema mais eficiente e fácil de administrar.